

UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

- **Curso: Fundamentos de Programación**
- **Docente:** Américo Chulluncuy Reynoso

2026-1

Sesión 12:

Archivos II

Contenido

1. Motivación: limitaciones del archivo de texto.
2. Memoria vs Disco: la conexión binaria
3. Archivos binarios
4. Structs como unidad de registro
5. Escritura binaria
6. Lectura binaria
7. Punteros de archivo: seekg, seekp, tellg, tellp
8. Acceso aleatorio
9. Ejercicio integrador: Sistema de estudiantes

1. Limitaciones del archivo de texto

```
double pi = 3.141592653589793;
```

- Los archivos de texto pueden perder precisión si no se escriben suficientes dígitos.

- Suelen ocupar mayor tamaño: En texto pi puede ocupar 17 bytes o más.

- Son lentos: double -> texto -> disco, hay conversiones

- No permiten acceso directo a registros ni guardar estructuras

```
struct Alumno {
    int codigo;
    char nombre[50];
    double promedio;
};
```

Texto vs Binario: la diferencia en bytes

Dato	En texto	En binario	Diferencia
`double` 3.14159265358979	16 bytes, con pérdida	8 bytes, exacto	2x más grande, menos preciso
`int` 1000000	7 bytes	4 bytes, exacto	casi 2x más grande
`struct` de 3 campos	variable, impredecible	`sizeof(struct)`, fijo	tamaño impredecible vs fijo
Arreglo de 1000 `int`	hasta 7000 bytes	4000 bytes exactos	hasta 75% más grande

- El **archivo binario** es simplemente copiar la memoria al disco tal como está, sin traducción.
- Es más rápido, más compacto y más preciso.
- La única desventaja: no puedes abrirlo con un editor de texto y leerlo.

2. Memoria vs Disco: Conexión binaria

- Cada tipo de dato ocupa un número fijo de bytes en memoria (sizeof). Hoy aprenderemos que esos bytes se pueden copiar directamente al disco.

```
cout << sizeof(char)   << endl;  // 1 byte
cout << sizeof(int)    << endl;  // 4 bytes
cout << sizeof(double) << endl;  // 8 bytes

struct Estudiante {
    char   nombre[50];  // 50 bytes
    int    codigo;      // 4 bytes
    double nota;        // 8 bytes
};
cout << sizeof(Estudiante) << endl;  // 62 bytes (o más por alineación)
```

3. ¿Cómo pensar en archivos binarios?

- Imagina que la RAM es una pizarra y el disco es una fotografía de esa pizarra. **write** toma la foto. **read** restaura la pizarra a partir de la foto.

```
int n = 255;
ofstream txt("numero.txt"); // 3 bytes al disco
txt << n;

// Archivo binario:
ofstream bin("numero.bin", ios::binary); // copia los 4 bytes al disco
bin.write(reinterpret_cast<char*>(&n), sizeof(n));
```

write recibe dos argumentos:

- la dirección de memoria desde donde copiar (char*) y
- cuántos bytes copiar.

Eso es todo lo que hace — copiar bytes.
La única pregunta que necesitamos responder es: ¿desde dónde? y ¿cuántos?

reinterpret_cast: la traducción del compilador

`reinterpret_cast<char*>(&p)`

toma la dirección de p y lo trata como una secuencia de bytes (char*)

```
int n = 42;
int* pInt = &n;   // puntero a int: "aquí hay un entero"

char* pBytes = reinterpret_cast<char*>(pInt);
// puntero a char: "aquí hay bytes" — misma dirección, distinta interpretación

// Ahora podemos ver los bytes individuales del entero
for (int i = 0; i < sizeof(int); i++)
    cout << "byte " << i << ": " << (int)pBytes[i] << endl;
```

Para escribir cualquier variable en binario:

```
archivo.write(reinterpret_cast<const char*>(&variable), sizeof(variable));
```

Para leer cualquier variable desde binario:

```
archivo.read(reinterpret_cast<char*>(&variable), sizeof(variable));
```

4. ¿Por qué string no funciona en archivos binarios?

- Un string en C++ no guarda el texto dentro de sí mismo. Guarda un puntero hacia una zona del heap donde vive el texto real. En memoria se ve así:

```
struct string {
    char*  ptr;   // apunta al heap — una dirección de memoria
    size_t size;
    size_t capacity;
};
```

- Si copias un string al disco con write, lo que grabas es la dirección de memoria — un número que solo tiene sentido en esa ejecución, en esa máquina, en ese momento. Al leerlo de vuelta, esa dirección apunta a basura o a memoria que ya no te pertenece.

Usar structs autocontenidos para archivos binarios

- Un struct para archivos binarios debe ser autocontenido: todos sus datos deben vivir dentro del struct, no en el heap. Eso significa usar char[] en lugar de string.

```
const int MAX_NOMBRE   = 50;
const int MAX_TELEFONO = 15;
struct Estudiante {
    int    codigo;                 // 4 bytes
    char   nombre[MAX_NOMBRE];     // 50 bytes — vive dentro del struct
    double nota;                   // 8 bytes
    bool   activo;                 // 1 byte (para eliminación lógica)
};
// sizeof(Estudiante) = fijo, predecible, autocontenido
// Todo el struct se puede copiar al disco en una sola operación
```

5. Escribiendo un registro binario

6. Leyendo registros binarios

Contando registros en un archivo binario

```
int contarRegistros(const char* nombreArchivo) {
    ifstream archivo(nombreArchivo, ios::binary);
    if (!archivo.is_open()) return 0;

    // mover al final del archivo
    archivo.seekg(0, ios::end);

    // posición actual = tamaño total en bytes
    long tamanioTotal = archivo.tellg();

    archivo.close();

    // número de registros = tamaño total / tamaño de un registro
    return tamanioTotal / sizeof(Estudiante);
}
```

Esta función usa seekg y tellg — dos herramientas para mover y consultar el puntero de lectura, podemos ver la utilidad inmediata: saber cuántos registros hay sin leerlos todos.

7. Punteros de archivo: posición en el disco

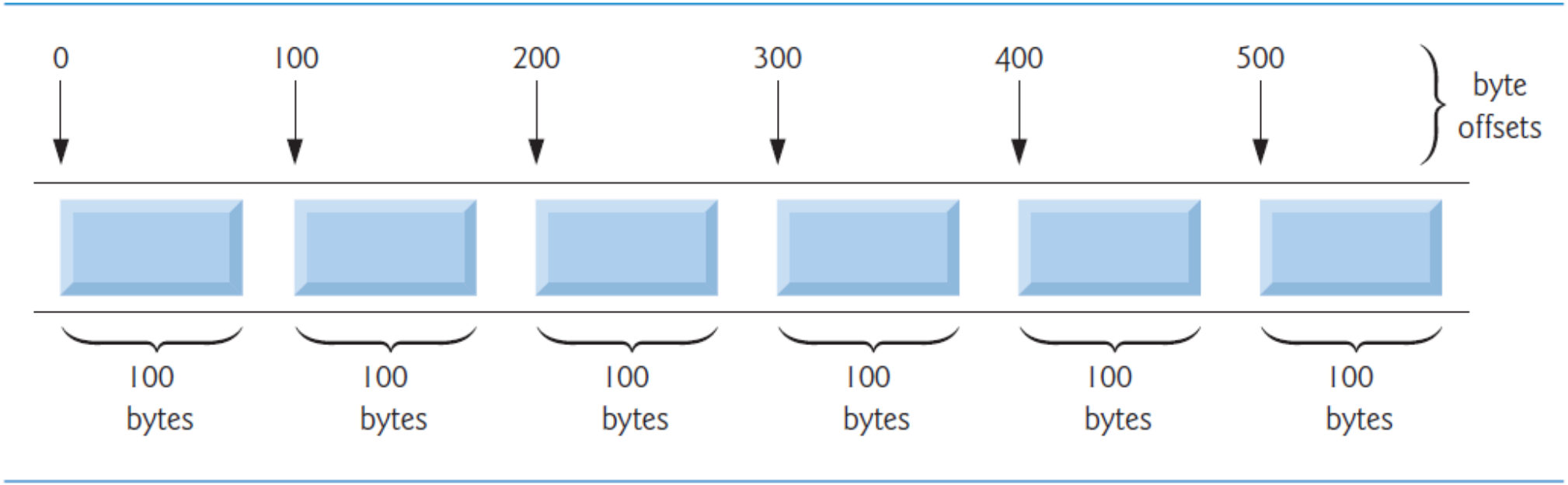
Función	Stream	¿Qué hace?
`seekg(pos)`	Entrada (`ifstream`)	Mueve la cabeza lectora a la posición `pos`
`seekg(offset, dir)`	Entrada	Mueve relativamente desde `dir`
`tellg()`	Entrada	Retorna la posición actual de la cabeza lectora
`seekp(pos)`	Salida (`ofstream`)	Mueve la cabeza escritora a la posición `pos`
`seekp(offset, dir)`	Salida	Mueve relativamente desde `dir`
`tellp()`	Salida	Retorna la posición actual de la cabeza escritora

Tres puntos de referencia para dir

```
ios::beg   // desde el inicio del archivo  (offset positivo)
ios::cur   // desde la posición actual     (offset + o -)
ios::end   // desde el final del archivo   (offset negativo)
```

8. Acceso aletorio: ir al registro directamente

- En un archivo de texto secuencial, para llegar al registro 500 hay que leer los 499 anteriores. En un archivo binario con registros de tamaño fijo, puedes ir directamente al registro 500 calculando su posición.



Para ir al registro N (base 0): archivo.seekg(N * sizeof(Estudiante), ios::beg);
Para ir al registro N (base 1): archivo.seekg((N-1) * sizeof(Estudiante), ios::beg);

8. Leer y escribir por posición

9. Ejercicio: Sistema de estudiantes

Desarrolla un sistema de gestión de estudiantes sobre un archivo binario con acceso aleatorio. El sistema debe permitir:

- **C**reate: agregar un nuevo estudiante al archivo
- **R**ead: mostrar todos los estudiantes activos / buscar por código
- **U**pdate: modificar la nota de un estudiante existente
- **D**elete: eliminación lógica: marcar como inactivo sin borrar del disco

  Explique cual es la ventaja de la eliminación lógica y no física.